

Algorithm Analysis

Algorithm analysis means figuring out how fast an algorithm runs

Computer scientists classify algorithm performance into three distinct scenarios: **best case, average case, and worst case**

Best Case = When everything goes perfectly

Average Case = What usually happens

Worst Case = When things go badly

Think of it like planning a trip:

- **Best case:** No traffic, green lights all the way
- **Average case:** Normal traffic, some delays
- **Worst case:** Accident, road closed, huge detour

Best Case - The Optimistic View

Best case = Minimum operations needed with perfect input

Example: Linear Search

- Looking for a number in a list
- Best case: The number is the FIRST item
- Only need 1 comparison!

Why best case isn't very useful:

- Rarely happens in real life
- Too optimistic - gives false confidence
- Doesn't tell you what to expect

Like assuming you'll always win the lottery - bad planning!

Worst Case - The Realistic View

Worst case = Maximum operations needed with any input

Example: Linear Search

- Looking for a number in a list of n items
- **Worst case:** Number is LAST or NOT there
- Need to check ALL n items!

Why worst case is what matters:

- Mathematically easier to calculate
- Gives you a guarantee (won't be slower than this)
- Critical for safety (medical devices, airplanes)
- What users care about (system shouldn't freeze)

Like budgeting for the worst traffic - you'll never be late!

Average Case - The Middle Ground

Average case = Expected operations with typical inputs

Example: Linear Search**

- Looking for a number in a random list
- On average: Check about $n/2$ items**
- If list has 100 items, average = 50 checks

The catch:

- You need to know what "typical" means
- Real data might not match your assumptions
- Harder to calculate mathematically

Like planning for "normal traffic" - but what's normal?*

Real-World Examples with Linear Search

Consider a linear search implementation: `for(int i = 0; i < n; i++) { if (arr[i] == target) return i; }`

Three Scenarios - Quick Summary

Linear Search Example (looking through 100 items)

Scenario	When t happens	How many checks
Best Case	Item is first	1 check
Average Case	Random location	~50 checks
Worst Case	Item is last or missing	100 checks

Bubble Sort Example (sorting 100 numbers)

Scenario	When it happens	Operations	
Best Case	Already sorted	~100 checks (fast!)	
Average Case	Random order	~5,000 checks	
Worst Case	Reverse order	~10,000 checks	

Why Worst Case Rules the World

In real systems, worst case is king:

Medical devices - Heart monitor MUST respond in time, every time

Flight control - Can't have the system slow down during landing

Gaming - Players notice when games lag

User experience - People hate waiting

Engineers design for worst case:

- If worst case takes 2 seconds, budget for 2 seconds
- If worst case takes 10 minutes, maybe pick a different algorithm

Hope for the best, plan for the worst!

Introduction to Big-O Notation

Big-O = A way to describe how fast an algorithm grows as input gets bigger

Think of it like:

- $O(1)$ = "I'm done instantly"
- $O(n)$ = "I'll finish when you stop typing"
- $O(n^2)$ = "Come back next week"

The key idea: We only care about what happens with LARGE inputs

- Who cares if it takes 1 second or 2 seconds with 10 items?

With 1 MILLION items, the difference matters!

Asymptotic Analysis

1-Big-O Notation (Upper Bounds)

$f(n) = O(g(n))$ means:

if there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. Intuitive Meaning: $f(n)$ grows no faster than $g(n)$ (asymptotically). We ignore constant factors and focus on the growth rate. "f(n) grows no faster than g(n) for large n"

What this really means:

- **Your actual code = $f(n)$ (complicated)**
- **Big-O simplification = $g(n)$ (simple)**

Example:

- Your code does: $3n + 2$ operations
- Big-O says: $O(n)$
- Why? The "3" and "+2" don't matter for large n
Like saying "millions" instead of "3,472,189" - close enough!

Real $f(n)$ vs Simplified $g(n)$

$g(n)$ is a much simpler mathematical function that captures only the dominant growth pattern of $f(n)$ as the input size n becomes very large

$f(n)$ = The actual messy truth

What your code really does (every operation counted):

| Algorithm | Actual operations $f(n)$ |

| Linear search | n (just count comparisons) |

| Better linear search | $3n + 5$ (comparisons + assignments) |

| Bubble sort | $n^2/2 - n/2 + 5$ |

$g(n)$ = The simplified version

What we actually use for analysis:

| $f(n)$ | Simplified to $g(n)$ |

| n | $O(n)$ |

| $3n + 5$ | $O(n)$ |

| $n^2/2 - n/2 + 5$ | $O(n^2)$ |

| $2^n + n^2$ | $O(2^n)$ |

Big-O Rules - Keep It Simple!

Rule 1: Drop the constants

- $4n^2 \rightarrow O(n^2)$
- $0.001n \rightarrow O(n)$
- $1000 \log n \rightarrow O(\log n)$

Rule 2: Keep the biggest term only

- $n^2 + n \rightarrow O(n^2)$ (n^2 wins!)
- $n^3 + 1000n^2 \rightarrow O(n^3)$ (n^3 wins!)
- $2^n + n^{50} \rightarrow O(2^n)$ (exponential beats polynomial!)

Rule 3: If it's constant, it's $O(1)$

- 7 operations $\rightarrow O(1)$
- 1000 operations $\rightarrow O(1)$
- 1,000,000 operations $\rightarrow$ Still $O(1)$!

Constants don't matter for large n - growth rate is everything!*

Common Complexity Classes - From Best to Worst

Ranked by speed (fastest to slowest):

Class	Name	Example	For n=1000
O(1)	Constant	Array lookup	1 operation
O(log n)	Logarithmic	Binary search	~10 ops
O(n)	Linear	Simple loop	1000 ops
O(n log n)	Linearithmic	Merge sort	~10,000 ops
O(n ²)	Quadratic	Bubble sort	1,000,000 ops
O(2 ⁿ)	Exponential	Brute force	10 ³⁰⁰ ops (impossible!)

Big-Omega (Ω) - The Lower Bound

Ω (Big-Omega) = The BEST it could possibly do

- A guarantee: "This algorithm takes AT LEAST this long"

Definition: $f(n) = \Omega(g(n))$ means $f(n)$ grows at least as fast as $g(n)$

Example:

- Linear search: $\Omega(1)$ (could find it first try)
- But it's also $\Omega(n)$? No - it's not ALWAYS taking n time

When we use Ω :

- Proving a problem is HARD
- Showing no algorithm can do better than X
- "Sorting can't be faster than $\Omega(n \log n)$ "

Think: "You can't finish this race in under 10 seconds"

3-Big-Theta (Θ) - The Tight Bound

Θ (Big-Theta) = The PERFECT match

- Upper bound AND lower bound are the same
- "This algorithm grows EXACTLY like this"

Definition: $f(n) = \Theta(g(n))$ means:

- $f(n) = O(g(n))$ AND
- $f(n) = \Omega(g(n))$

Example:

- $3n + 2 = \Theta(n)$
 - It's $O(n)$ (no faster than n)
 - It's $\Omega(n)$ (no slower than n)
 - So it's EXACTLY n -growth!

When to use Θ :

- When you really know the algorithm's behavior
- But most people just use O to be safe

How to Calculate Running Time

Step-by-step method:

- 1-Find the basic operation (usually the innermost loop)
- 2-Count how many times it runs
- 3- Add up counts from different parts
- 4- Simplify to Big-O

Example - Simple Loop:

```
```c  
int sum = 0;
for (int i = 0; i < n; i++)
 sum += A[i];
```
```

- Basic operation: array access + addition
- Runs: n times
- $T(n) = O(n)$

Example - Nested Loops

```
```c  

int count = 0;
for (int i = 0; i < n; i++)
 for (int j = i; j < n; j++)
 count++;
```
```

Let's count:

- $i=0$: j runs n times
- $i=1$: j runs $n-1$ times
- $i=2$: j runs $n-2$ times
- ...
- $i=n-1$: j runs 1 time

$$\begin{aligned}\text{Total} &= n + (n-1) + (n-2) + \dots + 1 \\ &= n(n+1)/2 \\ &= O(n^2)\end{aligned}$$

When in doubt, nested loops usually mean $O(n^2)$!

Example - Multiple Sections

```
```c
```

```
// Section A: $O(n)$
```

```
for (int i = 0; i < n; i++)
```

```
 doWork();
```

```
// Section B: $O(n^2)$
```

```
for (int i = 0; i < n; i++)
```

```
 for (int j = 0; j < n; j++)
```

```
 doOtherWork();
```

```
```
```

$$\text{Total} = O(n) + O(n^2)$$

Which one wins? $O(n^2)$!

Rule: The biggest term eats the smaller ones

- $O(n^2) + O(n) = O(n^2)$
- $O(n^3) + O(n^2) = O(n^3)$
- $O(2^n) + \text{anything} = O(2^n)$

Classifying Functions

Common algorithmic complexity classes, ordered from best to worst:

| Complexity | Name | Intuition | Example |
|---------------------------------|--------------|-------------------------------|--|
| $O(1)$ | Constant | Doesn't depend on input size | Array index access: <code>arr[5]</code> |
| $O(\log n)$ | Logarithmic | Halves problem each iteration | Binary search, balanced trees |
| $O(n)$ | Linear | Process each item once | Linear search, single loop |
| $O(n \log n)$ | Linearithmic | Divide-and-conquer sorting | Mergesort, heapsort, quicksort avg |
| $O(n^2)$ | Quadratic | Nested loops over same input | Bubble sort, insertion sort, brute-force |
| $O(n^3)$ | Cubic | Triple nested loops | Matrix multiplication (naive) |
| $O(2^n)$ | Exponential | Explores all subsets | Subset generation, brute-force NP |
| $O(n!)$ | Factorial | All permutations | Permutation generation |

How to Speed Up Programs - Strategy 1

#1: Choose a better algorithm (BIGGEST IMPACT!)

The difference algorithms make:

| n | $O(n)$ | $O(n^2)$ | $O(2^n)$ |

| 10 | 10 ops | 100 ops | 1,024 ops |

| 100 | 100 ops | 10,000 ops | 10^{30} ops (impossible) |

| 1000 | 1,000 ops | 1,000,000 ops | Universe ends |

Real example:

- Linear search $O(n)$ - fine for small lists
- Binary search $O(\log n)$ - much faster for large lists
- Switching from $O(n^2)$ to $O(n \log n)$ can make 1000x difference!

Best optimization = better algorithm, not tweaking code!

How to Speed Up Programs - Strategy 2

2-Remove redundant computation

Good:

```
```c
```

```
// Calculate once, reuse
```

```
int val = expensive_function(i);
```

```
for (int j = 0; j < n; j++)
```

```
 if (val == j) // Now just n times!
```

```
...
```

Good:

```
```c
// Calculate once, reuse
int val = expensive_function(i);
for (int j = 0; j < n; j++)
    if (val == j) // Now just n times!
...

```

Cache results - Store computed values for reuse

Why cook a meal 100 times when you can cook once and reheat?

How to Speed Up Programs - Strategy 3

3-Pick better data structures

Operation	Array	Linked List	Hash Table	Tree
-----------	-------	-------------	------------	------

Search by value	$O(n)$	$O(n)$	$O(1)$ avg	$O(\log n)$
-----------------	--------	--------	------------	-------------

Insert at end	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$
---------------	--------	--------	--------	-------------

Delete	$O(n)$	$O(n)$	$O(1)$ avg	$O(\log n)$
--------	--------	--------	------------	-------------

Choose based on what you do most:

- Lots of searches? → Hash table or tree
- Lots of inserts/deletes? → Linked list
- Need ordered data? → Tree
- Simple and fast? → Array

Right tool for the right job!

How to Speed Up Programs - Strategy 4

#4: System-level optimizations

Cache locality - Access memory that's close together

- Arrays = good (items next to each other)
- Linked lists = bad (items scattered)

Minimize I/O - Disk/network is SLOW

- Read once, use many times
- Batch operations together

Leverage hardware - Modern CPUs are smart

- Processors like predictable patterns
- Avoid branching if possible

These give you 2-10x speedup, not 1000x. Do these LAST!

Summary - Key Takeaways

1. Three scenarios matter:

- Best case (rarely useful)
- Average case (if you know the data)
- Worst case (what we usually use)

2. Big-O is about growth, not exact speed:

- $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$

3-Simplify rules:

- Drop constants ($100n \rightarrow O(n)$)
- Keep biggest term ($n^2 + n \rightarrow O(n^2)$)

4. To speed up programs:

- 1- Better algorithm (biggest win)
- 2- Remove redundant work
- 3- Better data structures
- 4- System tweaks (smallest win)

Symbol	Meaning	Plain English
$O(...)$	Upper bound	"No fast than..."
$\Omega(...)$	Lower bound	"At least this fast..."
$\Theta(...)$	Tight bound	"Exactly this fast..."

Complexity	Name	Example Use
------------	------	-------------

$O(1)$	Constant	Array access
--------	----------	--------------

$O(\log n)$	Logarithmic	Binary search
-------------	-------------	---------------

$O(n)$	Linear	Simple loop
--------	--------	-------------

$O(n \log n)$	Linearithmic	Merge sort
---------------	--------------	------------

$O(n^2)$	Quadratic	Bubble sort
----------	-----------	-------------

$O(2^n)$	Exponential	Brute force
----------	-------------	-------------

****Remember:**** For large n , growth rate is EVERYTHING!

Quick Reference Card

| Symbol | Meaning | Plain English |

| $O(\dots)$ | Upper bound | "No fast than..." |

| $\Omega(\dots)$ | Lower bound | "At least this fast..." |

| $\Theta(\dots)$ | Tight bound | "Exactly this fast..." |